Análise de Desempenho da Inferência de um Modelo de Linguagem Particionado em Múltiplas GPUs

Rafael Silva de Souza
Matheus Augusto Tregnago
Lucas Fraga Balbinot

Resumo

Modelos de linguagem de grande porte (Large Language Models – LLMs) tornaram-se fundamentais em aplicações modernas de inteligência artificial. Devido ao elevado número de parâmetros, esses modelos frequentemente não cabem na memória de uma única GPU, exigindo sua execução particionada em múltiplas GPUs.

Este trabalho propõe uma análise de desempenho da inferência de um modelo de linguagem distribuído, com foco no impacto do particionamento e da comunicação entre GPUs. O estudo considera as diferentes fases da inferência — *prefill* e *decode* — que apresentam características computacionais distintas.

Os experimentos serão conduzidos com o modelo Llama 3.1 8B no ambiente de computação de alto desempenho PCAD da UFRGS. Serão coletadas métricas de aplicação, como tempo até o primeiro token (TTFT), latência total (E2E), latência entre tokens (ITL) e vazão (TPS), além de métricas de GPU, como ocupação de multiprocessadores e uso de memória.

O objetivo é avaliar a escalabilidade do sistema, identificar gargalos de desempenho e compreender o impacto do paralelismo e da comunicação na inferência de LLMs em múltiplas GPUs.

1 Introdução

Modelos de linguagem de grande porte (LLMs) têm sido amplamente utilizados em aplicações como assistentes virtuais, tradução automática e geração de código [1]. Esses modelos possuem bilhões de parâmetros, o que impõe elevados requisitos computacionais mesmo durante a fase de inferência.

Um dos principais desafios na execução desses modelos é a limitação de memória das GPUs. Em muitos casos, o modelo completo não cabe em uma única GPU, sendo necessário particioná-lo entre múltiplas GPUs. Essa abordagem, embora viabilize a execução, introduz overhead de comunicação, que pode impactar significativamente o desempenho.

Além disso, a inferência em LLMs é composta por duas fases distintas: *prefill*, que processa o prompt de entrada de forma paralela, e *decode*, que gera tokens de forma sequencial e autorregressiva. Essas fases apresentam comportamentos computacionais distintos, o que torna a análise de desempenho mais complexa.

Neste contexto, este trabalho tem como objetivo analisar o desempenho da inferência de um modelo de linguagem distribuído em múltiplas GPUs, investigando o impacto do particionamento, da comunicação e das características de execução nas diferentes fases da inferência.

2 Objeto Computacional

O objeto de estudo deste trabalho é a execução da inferência do modelo de linguagem **Llama 3.1 8B**, que possui aproximadamente 8 bilhões de parâmetros e cerca de 16 GB de pesos.

Os experimentos serão conduzidos no ambiente de computação de alto desempenho (PCAD) da Universidade Federal do Rio Grande do Sul, que dispõe de múltiplos nós com GPUs de diferentes capacidades.

Serão utilizados três tipos de nós:

- **grace**: GPU NVIDIA L40S com 46 GB de memória, permitindo execução sem particionamento
- **tupi**: GPU RTX 4090 com 24 GB de memória
- **beagle**: duas GPUs GTX 1080 Ti com 11 GB cada, exigindo particionamento do modelo

A execução será realizada utilizando o framework **PyTorch** [3], com suporte à aceleração por GPU via CUDA. Operações críticas são executadas por meio da biblioteca **cuBLAS**, enquanto a comunicação entre múltiplas GPUs é realizada com **NCCL**.

3 Método de Análise

O método adotado baseia-se em medição experimental de desempenho da inferência do modelo em diferentes configurações de hardware e paralelismo.

Serão avaliados cenários com execução em uma única GPU e em múltiplas GPUs, com e sem particionamento do modelo. A análise considerará tanto o desempenho global quanto o comportamento das fases de *prefill* e *decode*.

A coleta de dados será realizada utilizando as seguintes ferramentas:

- **GenAI-Perf**: coleta de métricas como TTFT, ITL, TPS e latência total
- **NVIDIA Nsight Systems**: análise temporal da execução e identificação de kernels
- **NVIDIA Nsight Compute**: métricas de baixo nível como occupancy e uso de memória
- **nvidia-smi**: monitoramento de utilização de GPU, memória e consumo

Os experimentos serão conduzidos em ambiente controlado, com múltiplas execuções para garantir consistência estatística dos resultados.

4 Métricas de Desempenho

4.1 Métricas de Inferência

As principais métricas de desempenho da inferência incluem:

- **TTFT (Time to First Token)**: tempo entre o envio do prompt e a geração do primeiro token

- **E2E Latency**: tempo total para processar o prompt e gerar toda a resposta
- **ITL (Inter-Token Latency)**: tempo médio entre tokens consecutivos na fase de decode
- **TPS (Tokens per Second)**: número de tokens gerados por segundo

4.2 Métricas de GPU

Serão analisadas métricas relacionadas ao comportamento interno das GPUs:

- SM Occupancy
- uso de memória
- largura de banda de memória
- tempo de execução de kernels

5 Planejamento Experimental

Os experimentos serão conduzidos variando os seguintes parâmetros:

- número de GPUs utilizadas
- configuração de particionamento do modelo
- tamanho do prompt de entrada
- número de tokens gerados
- tamanho do batch

Cada configuração será executada múltiplas vezes, com o objetivo de reduzir a variabilidade e aumentar a confiabilidade dos resultados.

6 Resultados Esperados

Espera-se que a análise permita identificar gargalos de desempenho associados ao particionamento do modelo e à comunicação entre GPUs. Além disso, pretende-se avaliar a escalabilidade do sistema e compreender o impacto das diferentes fases da inferência no desempenho global.

Os resultados contribuirão para uma melhor compreensão da execução de modelos de linguagem em ambientes de computação de alto desempenho, auxiliando na escolha de estratégias eficientes de paralelização.

7 Uso de Ferramentas de Inteligência Artificial

Ferramentas de inteligência artificial foram utilizadas como apoio à elaboração deste trabalho.

Foi empregado um modelo de linguagem (ChatGPT, GPT-5.3, OpenAI) em regime interativo, com supervisão contínua dos autores. A ferramenta foi utilizada principalmente para apoio à redação, organização do conteúdo e esclarecimento de conceitos técnicos.

O uso seguiu um processo iterativo, envolvendo formulação de solicitações, análise crítica das respostas, validação pelos autores e adaptação do conteúdo antes da incorporação ao texto final.

Todo o conteúdo foi revisado pelos membros do grupo, garantindo correção técnica, consistência e reprodutibilidade.

Referências

[1] VASWANI, Ashish et al. Attention is all you need. 2017.
[2] BOMMASANI, Rishi et al. On the opportunities and risks of foundation models. 2021.
[3] PASZKE, Adam et al. PyTorch: An imperative style deep learning library. 2019.
[4] NVIDIA. CUDA C Programming Guide. 2023.